Experimento 5
Implementação de Circuitos Combinacionais com Multiplexadores

Grupo B2 - Turma T2
Allan Victor Almeida Faria, 261010546
Caio César Gonçalves Ribeiro, 211038217
Danilo Alves Ribeiro, 242003754

[1]Dep. Ciência da Computação – Universidade de Brasília (UnB)
CIC0231 - Laboratório de Circuitos Lógicos
16 de outubro de 2026

allanvictor.almeida@gmail.com, caiocesar20032003@gmail.com, daniloalvesjob@gmail.com

Abstract. *This report presents the design, synthesis and implementation on an FPGA of three combinational logic circuits described in a hardware description language (HDL): a full adder built from two 8-input multiplexers, a full adder built only from basic logic gates, and a special logic function built from a demultiplexer and a multiplexer. The circuits were synthesized in Quartus-II and recorded on the DE2-70 development kit. Functional and timing simulations, together with tests on the board that went through every input combination, matched the expected truth tables, which validated all three implementations.*

Resumo. *Este relatório apresenta o projeto, a síntese e a implementação em FPGA de três circuitos lógicos combinacionais descritos em linguagem de descrição de hardware (HDL): um somador completo construído com dois multiplexadores de 8 entradas, um somador completo construído apenas com portas lógicas básicas e uma função lógica especial construída a partir de um demultiplexador e de um multiplexador. Os circuitos foram sintetizados no Quartus-II e gravados no kit de desenvolvimento DE2-70. As simulações funcional e temporal, junto com os testes na placa percorrendo todas as combinações de entrada, concordaram com as tabelas-verdade esperadas, validando as três implementações.*

1. Introdução

Circuitos digitais combinacionais são aqueles cujas saídas dependem apenas da combinação atual das entradas, e podem ser descritos por tabelas-verdade e expressões booleanas [1]. Blocos como somadores, multiplexadores e demultiplexadores são a base de sistemas digitais mais complexos, como unidades aritméticas e circuitos de roteamento de dados.

Linguagens de descrição de hardware (HDL) permitem descrever esses circuitos de forma textual, simulá-los e sintetizá-los em dispositivos reconfiguráveis, como os FPGAs. Esse fluxo de projeto torna possível verificar o comportamento do circuito antes de gravá-lo e, depois, confirmá-lo na própria placa.

Neste experimento, foram implementados três circuitos: um somador completo construído com multiplexadores, um somador completo construído com portas lógicas tradicionais e uma função lógica especial construída com um demultiplexador e um multiplexador. Cada circuito foi validado por simulações funcional e temporal e por gravação no FPGA, percorrendo todas as combinações de entrada para comparar as saídas com as tabelas-verdade.

1.1. Objetivos

Projetar circuitos em linguagem de descrição de Hardware, sintetizando-os e implementando-os em FPGA.

1.2. Materiais

Neste experimento foram utilizados os seguintes materiais e equipamentos:

- Software Quartus-II versão 13.0 SP1
- Kit de desenvolvimento em FPGA DE2-70 Intel

2. Procedimentos e Resultados

Foram sintetizadas três funções lógicas a partir de código HDL para o DE2-70. Nesse contexto, gravou-se os testes iterando por cada combinação lógica, de modo a comprovar a corretude da implementação mediante comparação com a tabela-verdade das funções.

2.1. Somador com Multiplexador

A tabela verdade da saída normal e da saída carry de um somador completo são, respectivamente:

A	B	CIN	S	COUT
0	0	0	0	0
0	1	0	1	0
1	0	0	1	0
1	1	0	0	1
0	0	1	1	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	1

Figura 1. Somador Completo

Dessa forma, para construção de um somador a partir de dois multiplexadores de 8 entradas, um para cada saída, tomaremos A,B, Cin como bits de seleção, e fixaremos as entradas com os valores correspondentes aos valores esperados para um somador. Por exemplo, ao somarmos 1+1+1, esperamos que ambas as saídas estejam em 1. Como isso equivale a selecionar a sétima entrada em cada multiplexador, fixaremos 1 nessa respectiva entrada. Dessa forma, os códigos em HDL são:

```
module mux8(
    input [7:0] Dado,
    input [2:0] Escolha, // dadas
    output Saida
);
    assign Saida = Dado[Escolha];
endmodule
```

Figura 2. Mux8

```
module somador_mux(
    input  [2:0] SW,      // A=SW[2], B=SW[1], Cin=SW[0]
    output [1:0] LEDR     // Cout=LEDR[1], S=LEDR[0]
);
    mux8 mux_soma (
        .Dado    (8'b10010110),
        .Escolha (SW[2:0]),
        .Saida   (LEDR[0])
    );

    mux8 mux_carry (
        .Dado    (8'b11101000),
        .Escolha (SW[2:0]),
        .Saida   (LEDR[1])
    );
endmodule
```

Figura 3. Somador com Multiplexador

As simulações funcional e temporal mostraram a corretude de nosso modelo. Mais ainda, pode se ver a implementação no FPGA nesse link.:

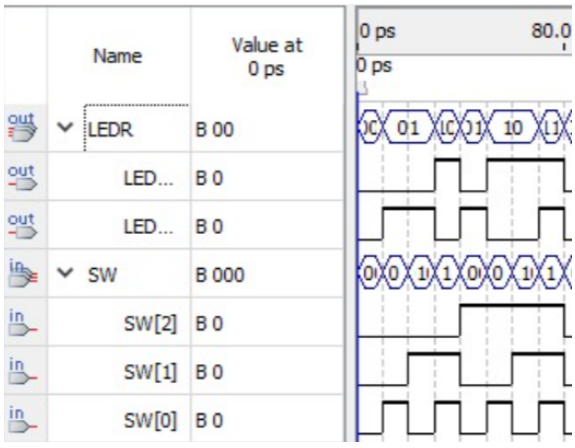


Figura 4. Simulação Funcional do Somador Com Mux

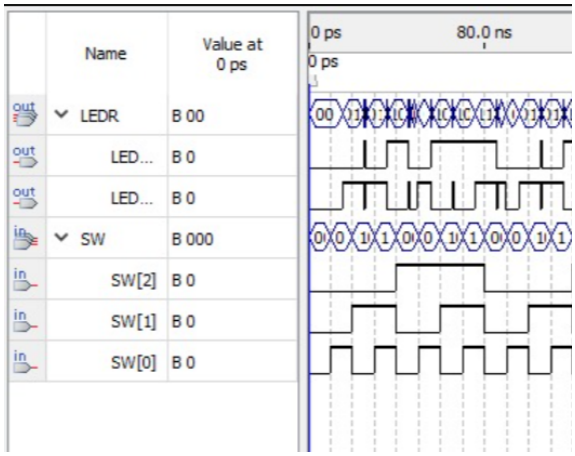


Figura 5. Simulação temporal do somador com Mux

2.2. Somador Normal

Implementa-se um somador completo de forma ”nativa”, usando somente portas lógicas tradicionais. A lógica se dá da seguinte forma: Para que a saída de carry seja ’1’, basta que duas das entradas sejam ’1’. Já para que a saída comum seja ’1’, o número de ’1’s nas entradas deve ser ímpar, o que é representado pela função XOR[1]. O código então é:

```
module somador_comp(
    input  [2:0] SW,       // A=SW[2], B=SW[1], Cin=SW[0]
    output [1:0] LEDR      // Cout=LEDR[1], S=LEDR[0]
);
    wire A   = SW[2];
    wire B   = SW[1];
    wire Cin = SW[0];

    assign {LEDR[1], LEDR[0]} = A + B + Cin;
endmodule
```

Figura 6. Còdigo do Somador Completo

O circuito foi sintetizado no FPGA, o que pode ser conferido nesse link. Também foram feitas as simulações, que mostraram resultado idêntico ao outro somador:

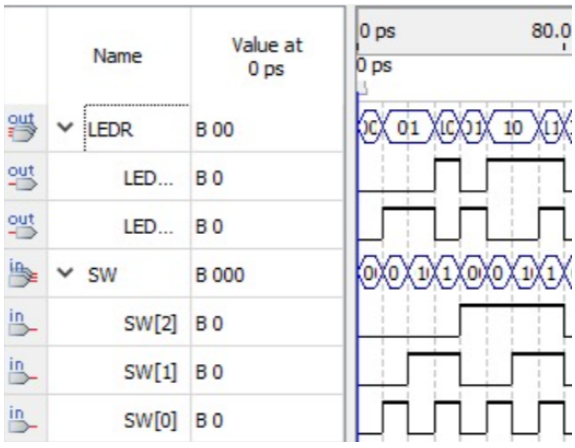


Figura 7. Simulação funcional do Somador

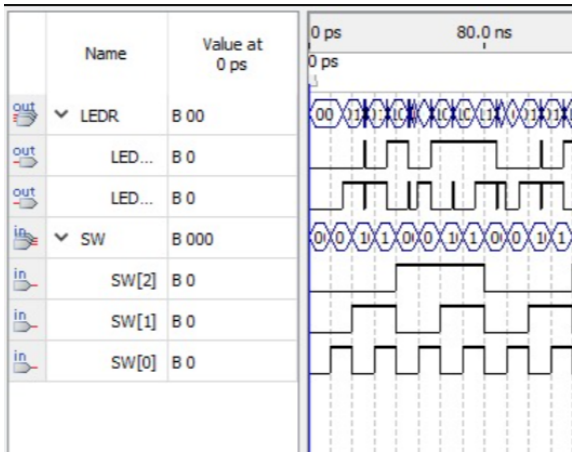


Figura 8. Simulação Temporal do Somador

2.3. Função Especial

Foi construída, a partir de um demultiplexador e do multiplexador, uma função lógica. O código HDL da função e do multiplexador é dado por:

```
module demux16(
    input Dado,
    input [3:0] Escolha,
    output [15:0] Saida
);
    genvar i;
    generate
        for (i = 0; i < 16; i = i + 1) begin : saidas
            assign Saida[i] = (Escolha == i) ? Dado : 1'b0;
        end
    endgenerate
endmodule
```

Figura 9. Código do Demultiplexador

```
module funcao7(
    input  [6:0] SW,       // {A,B,C,D,E,F,G} = SW[6:0]
    output [0:0] LEDR      // f = LEDR[0]
);
    wire [15:0] m;         // mintermos de ABCD

    demux16 decod (
        .Dado    (1'b1),
        .Escolha (SW[6:3]),   // {A,B,C,D}
        .Saida   (m)
    );

    wire [7:0] dados;
    assign dados[0] = 1'b0;
    assign dados[1] = m[0] | m[15];
    assign dados[2] = m[7];
    assign dados[3] = 1'b1;
    assign dados[4] = m[9] | m[15];
    assign dados[5] = 1'b0;
    assign dados[6] = m[10] | m[11];
    assign dados[7] = 1'b1;

    mux8 mux (
        .Dado    (dados),
        .Escolha (SW[2:0]),   // {E,F,G}
        .Saida   (LEDR[0])
    );
endmodule
```

Figura 10. Código da Função

Observou-se, tanto nas simulações, quanto no circuito sintetizado (cuja gravação pode ser vista aqui) a corretude do modelo, que respondeu ’1’ nos mintermos esperados. Atenta-se para a peridiocidade dos mintermos com FG, o termo mais abrangente da função:

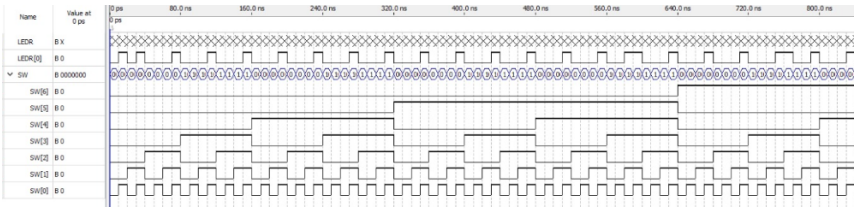


Figura 11. Simulação Funcional da Função

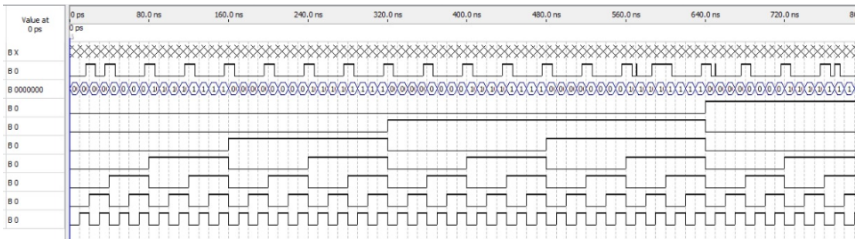


Figura 12. Simulação Temporal da Função

3. Conclusões

Os três circuitos projetados foram descritos em HDL, sintetizados e gravados com sucesso no FPGA DE2-70. Em todos os casos, as simulações funcional e temporal e os testes na placa, percorrendo todas as combinações de entrada, concordaram com as tabelas-verdade esperadas, o que comprova a corretude das implementações.

O somador com multiplexadores e o somador com portas lógicas apresentaram resultados idênticos, mostrando que uma mesma função pode ser implementada por estruturas diferentes. Já a função especial respondeu ’1’ exatamente nos mintermos esperados, incluindo a periodicidade associada ao termo FG.

Assim, o objetivo do experimento foi atingido: projetar circuitos em HDL, sintetizá-los e implementá-los em FPGA.

Referências

[1] John F. Wakerly. *Digital Design: Principles and Practices*. Pearson, 3rd edition, 1990.

Auto-Avaliação

1. v
2. v
3. f
4. v
5. v
6. v
7. v
8. f
9. v
10. f
11. v